

Problem Set 2: Solving Dynamic Oligopoly Games

Jordi Torres Vallverdú

May 1, 2026

EX 1

Instead of implementing encode/decode using binomial coefficients as in the MATLAB framework, I implement an equivalent bijection between state codes and weakly descending 3-tuples using dictionary enumeration in Julia¹. The function `states(kmax)` iterates over all valid tuples (w_1, w_2, w_3) with $w_1 \geq w_2 \geq w_3$ and assigns sequential integer codes. This produces the same bijection as the binomial approach but is more readable and easier to use in the different parts of the code. The decode table for $N = 3$, $\bar{w} = 3$ is shown below.

State code	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
w_1	0	1	1	1	2	2	2	2	2	2	3	3	3	3	3	3	3	3	3	3
w_2	0	0	1	1	0	1	1	2	2	2	0	1	1	2	2	2	3	3	3	3
w_3	0	0	0	1	0	0	1	0	1	2	0	0	1	0	1	2	0	1	2	3

Table 1: Decode table for $N = 3$, $\bar{w} = 3$. Column i represents the weakly descending 3-tuple encoded as state code i .

EX 2

The `calcv` function computes the continuation value for a firm moving up in efficiency (`val_up`) and staying at the same level (`val_stay`). It integrates over all possible rival investment outcomes. After the first contraction iteration with $N = 1$ and $\bar{w} = 19$:

After the first contraction iteration with $N = 1$ and $\bar{w} = 19$:

- Value function at $w = 10$: 1.262
- Value function at $w = 19$: 1.9008

Results are intuitive. Higher efficiency states have higher value as expected; a firm at $w = 19$ faces lower marginal cost and earns higher profits².

EX 3

The firm's value function is:

$$V_j = \pi_j + \beta [p \cdot V_{up} + (1 - p) \cdot V_{stay}] - x$$

¹I understand the Matlab code is general to handle $N > 3$ firms and combinatorial is more efficient the larger the state space. However, given that I am more familiar with Julia and in this language dictionaries are quite fast -also, given that we want to solve $N = 3$ -, I decided to code it this way to understand better the rest of the problem.

²The only meaningful difference from the MATLAB implementation is in initialization. For $N = 1$, MATLAB initializes over 20 states with values $1 + 0.1 \times (1, \dots, 20)$, ranging from 1.1 to 3.0. My code initializes over all 1540 states with $1 + 0.1 \times (1, \dots, 1540)$, ranging from 1.1 to 155. The contraction starts further from the fixed point, takes more iterations, and the first-iteration values in exercise2 differ as a result. The same issue goes to $N = 2$. The final $N = 3$ equilibrium converges correctly and produces consistent simulation results.

where $p = \frac{ax}{1+ax}$ is the probability of moving up. Taking the FOC with respect to x :

$$\beta\Delta V \cdot \frac{a}{(1+ax)^2} = 1$$

where $\Delta V = V_{up} - V_{stay}$. Solving for x^* :

$$x^* = \frac{\sqrt{a\beta\Delta V} - 1}{a}$$

If $\Delta V \leq 0$, then $x^* = 0^3$. The equilibrium selection criterion imposes that if firm i exits, all firms j with $w_j \leq w_i$ exit too.

EX 4

Starting from industry state $(6, 0, 0)$ and simulating forward 10,000 periods, Table 2 reports the average number of active firms and average total investment per period for both entry cost cases.

	High Entry Cost [0.15, 0.25]	Low Entry Cost [0.01, 0.11]
Average Active Firms	1.2634	1.8527
Average Total Investment	0.5891	0.5632

Table 2: Simulation Results: Baseline vs Low Entry Cost

With lower entry costs, significantly more firms are active on average (roughly, in different simulations I get $\approx 1.80 - 1.90$ vs $1.16 - 1.26$). Total investment remains roughly stable because the competition effect — each firm investing less due to lower returns from efficiency gains — offsets the more-firms effect. Investment per firm falls from $\approx 0.46 - 0.60$ to $\approx 0.31 - 0.50$ with lower entry costs, depending on the simulation.

³In the code I force this to be either 0 or Δ to avoid taking $\sqrt{\cdot}$ of negative values.